

LAPORAN AKHIR PRAKTIKUM

Mata Praktikum : Algoritma Pemrograman 2B
Kelas : IIA04
Praktikum ke- : 4
Tanggal : 26 April 2026
Materi : Flask
NPM : 50425612
Nama : Maulana Abdul Aziz
Ketua Asisten : Ni Gusti Ayu
Jumlah Lembar : 3



LABORATORIUM TEKNIK INFORMATIKA
UNIVERSITAS GUNADARMA
2026

1. Jelaskan logika program ACT yang dibuat!

Secara garis besar, program ini merupakan aplikasi REST API berbasis Flask yang dirancang untuk mengelola data buku. Alih-alih menggunakan database fisik seperti MySQL atau PostgreSQL, sistem ini mengimplementasikan metode in-memory storage, di mana data hanya disimpan sementara pada memori program menggunakan variabel bernama `datas`.

Sistem kerja program ini dapat dibagi menjadi dua komponen utama:

1. Mekanisme Penyimpanan dan Pemformatan

Database Simulasi: Variabel `datas` (berupa list of dictionaries) berfungsi sebagai tempat penampungan data. Karena sifatnya yang volatil, seluruh data yang diinput akan hilang secara otomatis apabila server di-restart.

Pemformatan Mata Uang: Terdapat fungsi helper `format_rupiah` yang bertugas mengonversi nilai integer (contoh: 105000) menjadi format string mata uang yang baku (contoh: Rp105.000).

2. Alur Logika Endpoint

- `GET /book` (Menampilkan Semua Buku): Saat menerima request, program melakukan iterasi pada seluruh data buku yang ada. Harga tiap buku akan diformat melalui `format_rupiah`, dimasukkan ke list baru (`formatted_datas`), dan dikembalikan sebagai response berformat JSON.
- `GET /book/<id>` (Mencari Buku Spesifik): Menggunakan ID yang diambil dari URL, program menjalankan pencarian sekuensial pada variabel `datas`. Jika sesuai, data buku akan ditampilkan. Jika ID tidak ditemukan, sistem akan merespons dengan status code 404 (Not Found).
- `POST /book` (Menambahkan Buku): Endpoint ini menerapkan validasi JSON yang ketat sebelum menerima data. Program memverifikasi keberadaan key `id` dan `items`, memastikan `items` berjenis list, mengecek kelengkapan atribut (`title`, `author`, `price`), serta memastikan `price` bernilai angka. Jika validasi gagal, server merespons dengan 400 (Bad Request). Jika berhasil, data ditambahkan (`append`) ke `datas` dengan status 201 (Created).
- `DELETE /book/<id>` (Menghapus Buku): Proses ini mencari posisi indeks dari ID yang di-request menggunakan fungsi `enumerate()`. Setelah indeks ditemukan, data tersebut dihapus dari sistem dengan perintah `del`.
- `GET /` (Halaman Utama): Endpoint root yang bertugas menampilkan pesan sapaan sederhana sebagai indikator bahwa server berjalan normal.

2. Jelaskan perbandingan Flask dengan framework web Python lainnya, khususnya dalam hal fleksibilitas dan kompleksitas

1. Fleksibilitas

Flask

Flask hanya menyediakan fondasi minimal yang mencakup sistem routing, penanganan request/response, serta mesin templat Jinja2. Framework ini tidak mewajibkan penggunaan lapisan basis data tertentu, mekanisme autentikasi, atau struktur proyek yang kaku. Pengembang diberikan kebebasan penuh untuk mengintegrasikan pustaka eksternal seperti SQLAlchemy (ORM), WTForms (validasi formulir), atau sistem autentikasi buatan sendiri. Karakteristik ini membuat Flask dapat diterapkan pada spektrum proyek yang luas, mulai dari API sederhana hingga aplikasi monolitik berskala besar.

Django

Django hadir dengan pendekatan sebaliknya, yaitu menyertakan hampir seluruh komponen yang umum diperlukan secara bawaan (`batteries-included`): ORM, autentikasi, panel admin, migrasi basis data, serta sistem formulir. Konsekuensinya, tingkat fleksibilitas Django lebih rendah dibandingkan Flask. Meskipun secara teknis beberapa komponen bawaan dapat diganti (misalnya mengganti mesin templat dengan Jinja2), proses tersebut membutuhkan usaha yang tidak sepele dan umumnya tidak direkomendasikan karena melawan arsitektur yang telah dirancang secara integratif.

FastAPI

FastAPI menawarkan fleksibilitas yang setara dengan Flask, namun dengan tambahan fitur modern seperti dukungan `async/await`, `dependency injection`, serta validasi data berbasis `type hints` Python (menggunakan `Pydantic`). Pengembang tetap bebas memilih komponen pelengkap, sementara FastAPI sendiri memberikan keuntungan tambahan berupa pembangkitan dokumentasi interaktif (Swagger UI dan ReDoc) secara otomatis. Fleksibilitas ini menjadikan FastAPI sangat sesuai untuk proyek yang mengutamakan performa tinggi dan spesifikasi antarmuka API yang ketat.

2. Kompleksitas

Flask

Kompleksitas awal Flask tergolong rendah. Aplikasi sederhana dapat diwujudkan hanya dalam beberapa baris kode tanpa struktur proyek yang ditentukan. Namun, seiring bertambahnya kebutuhan fungsional, pengembang harus secara mandiri mengintegrasikan dan mengatur berbagai pustaka tambahan. Hal ini berpotensi meningkatkan kompleksitas pengelolaan kode secara signifikan, terutama pada proyek berskala besar, karena tidak adanya pedoman arsitektur baku yang disediakan oleh framework.

Django

Sebaliknya, Django memiliki kompleksitas awal yang lebih tinggi. Untuk membangun aplikasi dasar sekalipun, pengembang perlu memahami konsep-konsep seperti proyek versus aplikasi, berkas pengaturan, pemetaan URL, model, tampilan, dan templat yang telah terstruktur secara baku. Akan tetapi, untuk proyek berukuran menengah hingga besar, Django justru mengurangi

kompleksitas jangka panjang. Keseragaman struktur, konvensi penamaan yang konsisten, serta integrasi erat antarkomponen bawaan membuat pemeliharaan lebih terprediksi dan efisien.

FastAPI

FastAPI berada pada posisi menengah dalam hal kompleksitas. Implementasi API sederhana sangat ringkas dan mudah dipahami karena memanfaatkan type hints secara jelas. Namun, pemanfaatan fitur lanjutan seperti dependency injection bertingkat, tugas latar belakang, atau komunikasi WebSocket menuntut pemahaman yang matang tentang pemrograman asinkron serta skema data Pydantic. Meskipun FastAPI tidak memaksakan struktur proyek tertentu (mirip Flask), keberadaan fitur validasi otomatis dan dokumentasi interaktif membantu pengembang mengendalikan kompleksitas lebih baik dibandingkan dengan Flask murni.